Python Engineering - EX22

WEEK 04 | CLASS 14 - File I/O | Vertical State Persistence

Goal: generate a small trajectory with a `VerticalState` object, save it to a CSV file, read it back, and verify that the reconstructed data matches the original.

> For the first time, our results will survive after Python stops: we move from temporary RAM data to persistent data stored on disk.

Exact path

```
01_python_bootcamp/
`-- week_04_oop_files_testing/
    `-- class_14_file_io/
        |-- notes/
        `-- exercises/
            `-- ex22_vertical_state_file_io/
                |-- vertical_state.py
                |-- file_io.py
                |-- main.py
                `-- data/
                    |-- vertical_trajectory.csv
                    `-- run_log.txt
```

Exercise architecture

```
vertical_state.py -> defines engineering object
main.py           -> creates and propagates trajectory
file_io.py        -> writes and reads files
data/             -> persistent outputs
```

Reference values

Parameter	Value
Initial state	h=100.0 m, v=20.0 m/s, t=0.0 s, g=9.81 m/s^2
dt	0.1 s
n_steps	5
Saved rows	6 (initial + 5 propagated states)
Trajectory shape	(6, 3), columns [h, v, t]
Final Euler state	h=109.019 m, v=15.095 m/s, t=0.5 s approx.
Round-trip check	np.allclose(original, loaded) -> True

BLOCK A - Reuse VerticalState

Copy the cleaned EX21 class into `vertical_state.py`.

```
import numpy as np

class VerticalState:
    def __init__(self, h, v, t=0.0, g=9.81):
        self.h = h
        self.v = v
        self.t = t
        self.g = g

    def state_vector(self):
        return np.array([self.h, self.v, self.t])

    def euler_step(self, dt):
        self.h = self.h + self.v * dt
        self.v = self.v - self.g * dt
        self.t = self.t + dt
```

BLOCK B - Build a robust path with pathlib

```
from pathlib import Path

base_dir = Path(__file__).resolve().parent
data_dir = base_dir / 'data'
data_dir.mkdir(exist_ok=True)

trajectory_file = data_dir / 'vertical_trajectory.csv'
log_file = data_dir / 'run_log.txt'
```

`__file__` represents the current Python file. `.parent` gives its directory. Path's `/` operator builds paths without manual string concatenation.

BLOCK C - Generate the trajectory in memory

```
earth_state = VerticalState(h=100.0, v=20.0)
dt = 0.1
n_steps = 5

trajectory_rows = [earth_state.state_vector()]

for _ in range(n_steps):
    earth_state.euler_step(dt)
    trajectory_rows.append(earth_state.state_vector())

trajectory = np.array(trajectory_rows)
```

Expected shape: `(6, 3)`.

BLOCK D - file_io.py: save CSV

```
def save_trajectory_csv(file_path, trajectory):

    with open(file_path, 'w', encoding='utf-8') as file:
        file.write('h_m,v_m_s,t_s\n')

        for row in trajectory:
            h = row[0]
            v = row[1]
            t = row[2]

            file.write(
                f'{h:.6f},{v:.6f},{t:.6f}\n'
            )
```

Mode 'w': opens for writing. Existing content is replaced. The `with` block ensures the file is closed correctly.

BLOCK E - Inspect the file

```
h_m,v_m_s,t_s
100.000000,20.000000,0.000000
102.000000,19.019000,0.100000
103.901900,18.038000,0.200000
105.705700,17.057000,0.300000
107.411400,16.076000,0.400000
109.019000,15.095000,0.500000
```

BLOCK F - file_io.py: load CSV

```
import numpy as np

def load_trajectory_csv(file_path):

    rows = []

    with open(file_path, 'r', encoding='utf-8') as file:
        header = file.readline()

        for line in file:
            clean_line = line.strip()

            if clean_line == '':
                continue

            values = clean_line.split(',')
            h = float(values[0])
            v = float(values[1])
            t = float(values[2])
            rows.append([h, v, t])

    return np.array(rows)

file text -> line string -> strip() -> split(',')
          -> numeric strings -> float() -> rows -> np.array
```

BLOCK G - Validate the round trip

```
save_trajectory_csv(trajectory_file, trajectory)
loaded_trajectory = load_trajectory_csv(trajectory_file)

same_shape = trajectory.shape == loaded_trajectory.shape
same_values = np.allclose(trajectory, loaded_trajectory)

print(same_shape)   # True
print(same_values)  # True
```

This is **round-trip validation**: data -> file -> reconstructed data.

BLOCK H - Practice append mode 'a'

```
def append_run_log(file_path, message):

    with open(file_path, 'a', encoding='utf-8') as file:
        file.write(message + '\n')

final_state = trajectory[-1]

message = (
    f'steps={n_steps}, dt={dt}, '
    f'final_h={final_state[0]:.6f}, '
    f'final_v={final_state[1]:.6f}, '
    f'final_t={final_state[2]:.6f}'
)

append_run_log(log_file, message)
```

Every execution adds a new line instead of replacing previous ones.

BLOCK I - Imports and main.py

```
import numpy as np
from pathlib import Path

from vertical_state import VerticalState
from file_io import (
    save_trajectory_csv,
    load_trajectory_csv,
    append_run_log
)

def main():
    # paths -> object -> trajectory -> save -> load
    # -> validate -> append log -> print summary

if __name__ == '__main__':
    main()
```

FINAL VALIDATION

```
Explain in comments:
- What is file I/O?
- RAM data vs persistent file data?
- What does open() return?
- What do modes 'w', 'r', and 'a' mean?
- Why use with open(...) as file?
- What does file.write() receive?
- Why do numbers become text in this CSV?
- What does file.readline() do?
- Why strip() before split(',')?
- What does split(',') return?
- Why convert with float()?
- What does Path(__file__).resolve().parent mean?
- Why separate file_io.py from vertical_state.py?
- What is round-trip validation?
- Why use np.allclose() for floating-point validation?
```

Completion criterion

EX22 is complete when you can explain the entire flow **object -> RAM trajectory -> CSV text -> read text -> numbers -> NumPy array** and understand the responsibilities of each module.